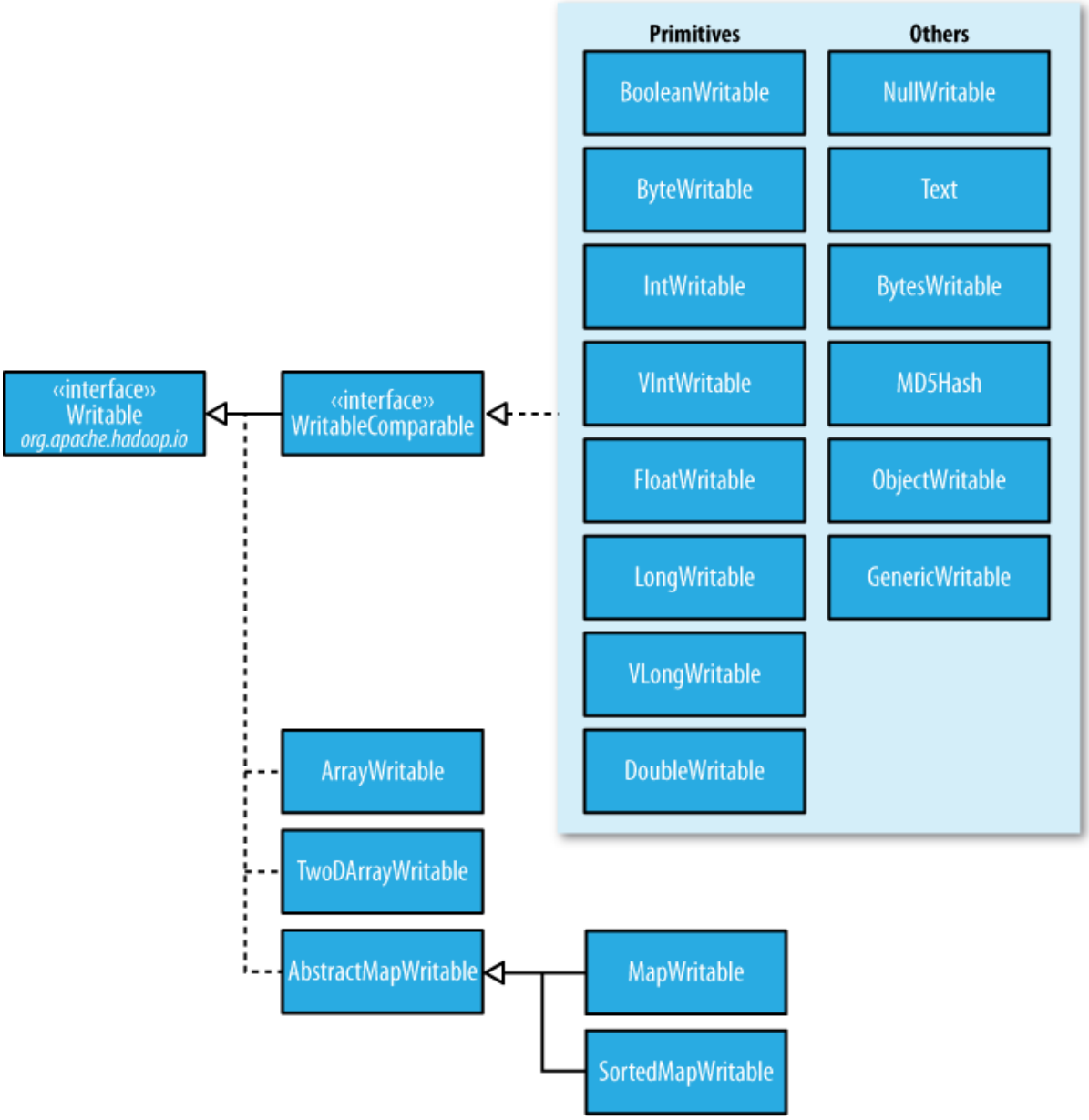


Writable Interface

```
package org.apache.hadoop.io;
import java.io.DataOutput;
import java.io.DataInput;
import java.io.IOException;
public interface Writable
{
void write(DataOutput out) throws IOException;
void readFields(DataInput in) throws IOException;
}
```

MapReduce is a programming framework that allows us to perform distributed and parallel processing on large data sets in a distributed environment.

- MapReduce consists of two distinct tasks – Map and Reduce.
- As the name MapReduce suggests, the reducer phase takes place after the mapper phase has been completed.
- So, the first is the map job, where a block of data is read and processed to produce key-value pairs as intermediate outputs.
- The output of a Mapper or map job (key-value pairs) is input to the Reducer.
- The reducer receives the key-value pair from multiple map jobs.
- Then, the reducer aggregates those intermediate data tuples (intermediate key-value pair) into a smaller set of tuples or key-value pairs which is the final output.

MapReduce majorly has the following three Classes. They are,

Mapper Class

The first stage in Data Processing using MapReduce is the **Mapper Class.** Here, RecordReader processes each Input record and generates the respective key-value pair. Hadoop's Mapper store saves this intermediate data into the local disk.

- **Input Split**

It is the logical representation of data. It represents a block of work that contains a single map task in the MapReduce Program.

- **RecordReader**

It interacts with the Input split and converts the obtained data in the form of **Key-Value** Pairs.

Reducer Class

The Intermediate output generated from the mapper is fed to the reducer which processes it and generates the final output which is then saved in the **HDFS.**

Driver Class

The major component in a MapReduce job is a **Driver Class.**

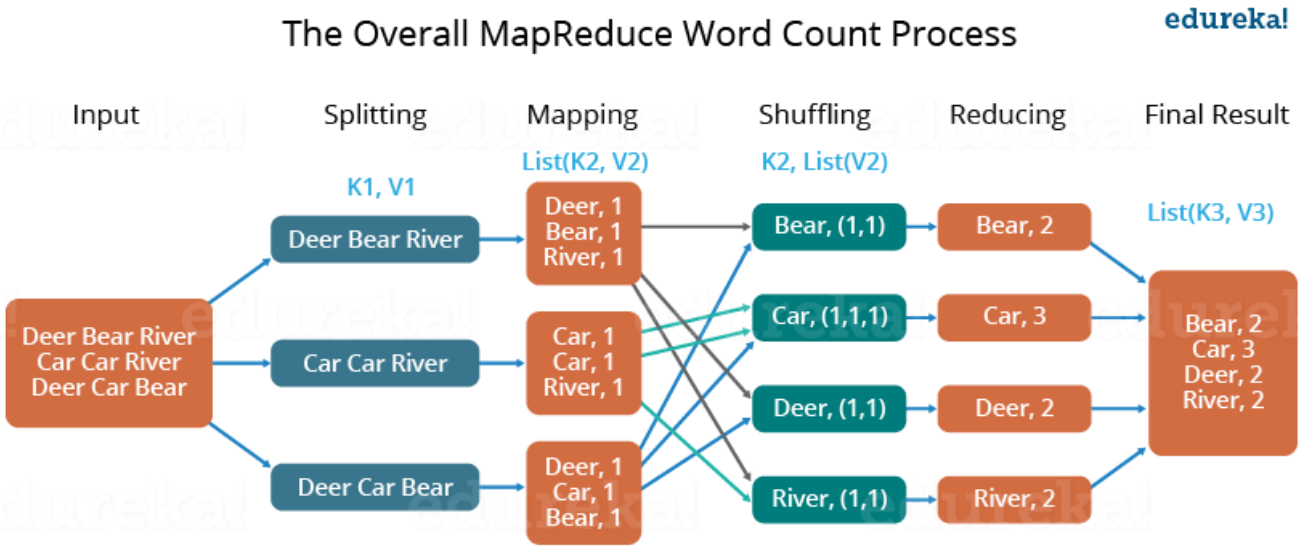
It is responsible for setting up a MapReduce Job to run-in Hadoop.

MapReduce Tutorial: A Word Count Example of MapReduce

Let us understand, how a MapReduce works by taking an example where I have a text file called example.txt whose contents are as follows:

Dear, Bear, River, Car, Car, River, Deer, Car and Bear

Now, suppose, we have to perform a word count on the sample.txt using MapReduce. So, we will be finding the unique words and the number of occurrences of those unique words.



- First, we divide the input into three splits as shown in the figure. This will distribute the work among all the map nodes.
- Then, we tokenize the words in each of the mappers and give a hardcoded value (1) to each of the tokens or words. The rationale behind giving
-
- a hardcoded value equal to 1 is that every word, in itself, will occur once.
- Now, a list of key-value pair will be created where the key is nothing but the individual words and value is one. So, for the first line (Dear Bear River) we have 3 key-value pairs – Dear, 1; Bear, 1; River, 1. The mapping process remains the same on all the nodes.
- After the mapper phase, a partition process takes place where sorting and shuffling happen so that all the tuples with the same key are sent to the corresponding reducer.
- So, after the sorting and shuffling phase, each reducer will have a unique key and a list of values corresponding to that every key. For example, Bear, [1,1]; Car, [1,1,1].., etc.

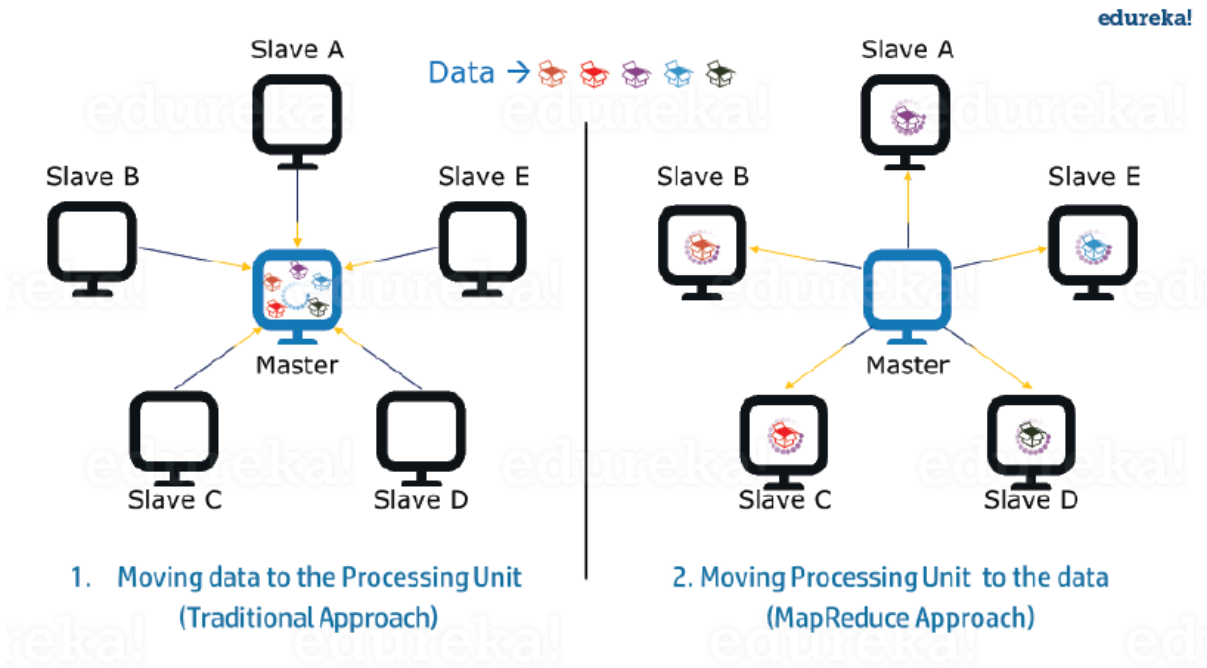
- Now, each Reducer counts the values which are present in that list of values. As shown in the figure, reducer gets a list of values which is [1,1] for the key Bear. Then, it counts the number of ones in the very list and gives the final output as – Bear, 2.
- Finally, all the output key/value pairs are then collected and written in the output file.

MapReduce Tutorial: Advantages of MapReduce

The two biggest advantages of MapReduce are:

1. **Parallel Processing:**

In MapReduce, we are dividing the job among multiple nodes and each node works with a part of the job simultaneously. So, MapReduce is based on Divide and Conquer paradigm which helps us to process the data using different machines. As the data is processed by multiple machines instead of a single machine in parallel, the time taken to process the data gets reduced by a tremendous amount as shown in the figure below (2).



2. Data Locality:

Instead of moving data to the processing unit, we are moving the processing unit to the data in the MapReduce Framework. In the traditional system, we used to bring data to the processing unit and process it. But, as the data grew and became very huge, bringing this huge amount of data to the processing unit posed the following issues:

- Moving huge data to processing is costly and deteriorates the network performance.
- Processing takes time as the data is processed by a single unit which becomes the bottleneck.
- The master node can get over-burdened and may fail.

Now, MapReduce allows us to overcome the above issues by bringing the processing unit to the data. So, as you can see in the above image that the data is distributed among multiple nodes where each node processes the part of the data residing on it. This allows us to have the following advantages:

- It is very cost-effective to move processing unit to the data.
- The processing time is reduced as all the nodes are working with their part of the data in parallel.
- Every node gets a part of the data to process and therefore, there is no chance of a node getting overburdened.

MapReduce Program

The entire MapReduce program can be fundamentally divided into three parts:

- Mapper Phase Code
- Reducer Phase Code
- Driver Code

We will understand the code for each of these three parts sequentially.

Mapper code:

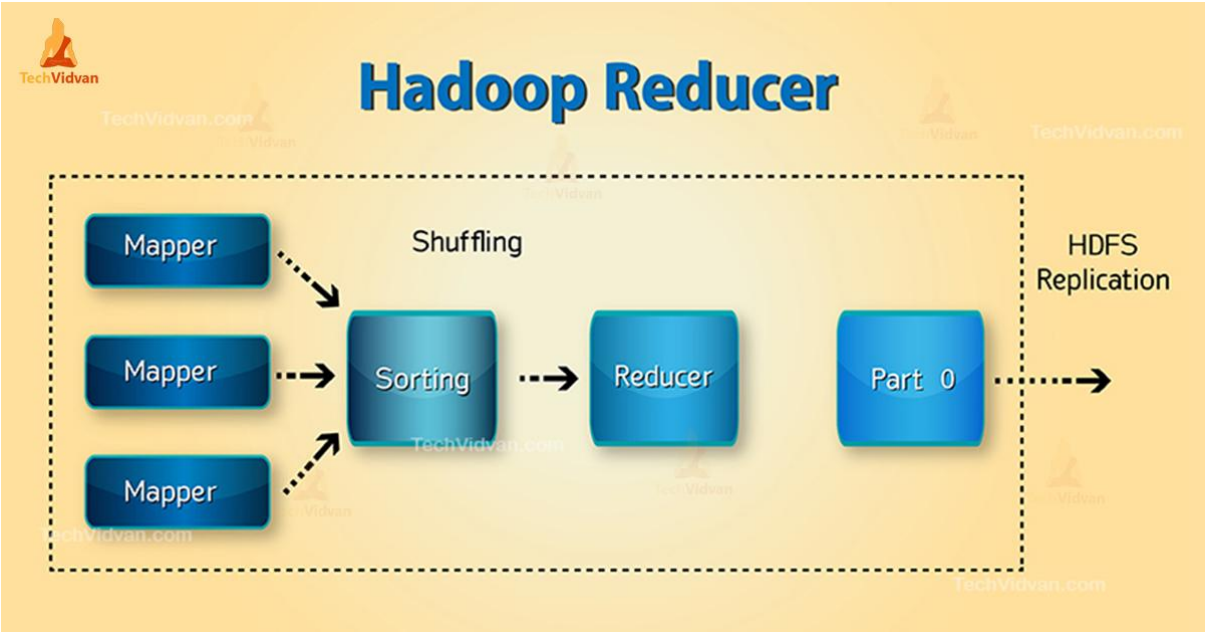
```
public class WordCount{
    public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{
       private final static IntWritable one = new IntWritable(1);
       private Text word = new Text();
       public void map(Object key, Text value, Context context
                    ) throws IOException, InterruptedException {
         StringTokenizer itr = new StringTokenizer(value.toString());
         while (itr.hasMoreTokens()) {
             word.set(itr.nextToken());
             context.write(word, one);
         }
      }
    }
```

- We have created a class Map that extends the class Mapper which is already defined in the MapReduce Framework.

Input Text File

Key	Value
0	Dear Bear River
121	Car Car River
226	Deer Car Bear

- We define the data types of input and output key/value pair after the class declaration using angle brackets.
- Both the input and output of the Mapper is a key/value pair.
- Input:
    - The *key* is nothing but the offset of each line in the text file: *LongWritable*
    - The *value* is each individual line
- Output:
    - The *key* is the tokenized words: *Text*
    - We have the hardcoded *value* in our case which is 1: *IntWritable*
    - Example – Dear 1, Bear 1, etc.
- We have written a java code where we have tokenized each word and assigned them a hardcoded value equal to *1*.



Reducer Code:

```
public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
        private IntWritable result = new IntWritable();
        public void reduce(Text key, Iterable<IntWritable> values,
                         Context context)
                       throws IOException, InterruptedException {
           int sum = 0;
           for (IntWritable val : values) {
                 sum += val.get();
           }
           result.set(sum);
           context.write(key, result);
        }
     }
```

- We have created a class Reduce which extends class Reducer like that of Mapper.
- We define the data types of input and output key/value pair after the class declaration using angle brackets as done for Mapper.
- Both the input and the output of the Reducer is a key-value pair.
- Input:
    - The *key* nothing but those unique words which have been generated after the sorting and shuffling phase: *Text*
    - The *value* is a list of integers corresponding to each key: *IntWritable*
    - Example – Bear, [1, 1], etc.
- Output:
    - The *key* is all the unique words present in the input text file: *Text*
    - The *value* is the number of occurrences of each of the unique words: *IntWritable*
    - Example – Bear, 2; Car, 3, etc.
- We have aggregated the values present in each of the list corresponding to each key and produced the final answer.
- In general, a single reducer is created for each of the unique words, but, you can specify the number of reducer in mapred-site.xml.

Driver Code:

```
public static void main(String[] args) throws Exception {
          Configuration conf = new Configuration();
          Job job = Job.getInstance(conf, "word count");
          job.setJarByClass(WordCount.class);
          job.setMapperClass(TokenizerMapper.class);
          job.setCombinerClass(IntSumReducer.class);
          job.setReducerClass(IntSumReducer.class);
          job.setOutputKeyClass(Text.class);
          job.setOutputValueClass(IntWritable.class);
          FileInputFormat.addInputPath(job, new Path(args[0]));
          FileOutputFormat.setOutputPath(job, new Path(args[1]));
          System.exit(job.waitForCompletion(true) ? 0 : 1);
```

```
  }
```

- In the driver class, we set the configuration of our MapReduce job to run in Hadoop.
- We specify the name of the job, the data type of input/output of the mapper and reducer.
- We also specify the names of the mapper and reducer classes.
- The path of the input and output folder is also specified.
- The method setInputFormatClass () is used for specifying how a Mapper will read the input data or what will be the unit of work. Here, we have chosen TextInputFormat so that a single line is read by the mapper at a time from the input text file.
- The main () method is the entry point for the driver. In this method, we instantiate a new Configuration object for the job

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount{
    public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{
       private final static IntWritable one = new IntWritable(1);
       private Text word = new Text();
       public void map(Object key, Text value, Context context
                   ) throws IOException, InterruptedException {
         StringTokenizer itr = new StringTokenizer(value.toString());
         while (itr.hasMoreTokens()) {
             word.set(itr.nextToken());
             context.write(word, one);
         }
       }
    }

    public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
        private IntWritable result = new IntWritable();
        public void reduce(Text key, Iterable<IntWritable> values,
                         Context context)
                       throws IOException, InterruptedException {
           int sum = 0;
           for (IntWritable val : values) {
                 sum += val.get();
           }
           result.set(sum);
           context.write(key, result);
        }
     }

     public static void main(String[] args) throws Exception {
          Configuration conf = new Configuration();
          Job job = Job.getInstance(conf, "word count");
          job.setJarByClass(WordCount.class);
          job.setMapperClass(TokenizerMapper.class);
          job.setCombinerClass(IntSumReducer.class);
          job.setReducerClass(IntSumReducer.class);
          job.setOutputKeyClass(Text.class);
          job.setOutputValueClass(IntWritable.class);
          FileInputFormat.addInputPath(job, new Path(args[0]));
          FileOutputFormat.setOutputPath(job, new Path(args[1]));
          System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

RecordReader

- Apache Hadoop can process any arbitrary data like log files, text files, structured data etc. We know that actual data is stored in HDFS while InputSplit is the logical partition of the data.
- InputSplit is the chunk of data that processed by single map (i.e. one Mapper can process one InputSplit at a time) Now each split is required to divide into the records. Please note that inputsplit do not contain actual data rather they contain the references to the actual data (HDFS blocks).
- InputFormat is responsible for validating the input data, creating the inputsplit and divide them into the records.
- RecordReader is the class which actually loads the data from Source. Mapper always understands key value pair, so Record Reader reads line by line from input split and converts it into key value pair for the mapper.
- RecordReader uses the data within the boundaries, defined by InputSplit. It creates key-value pairs for the mapper. The >“start” is the byte position in the file, thus RecordReader starts generating key-value at ‘start. And the “end” is where it should stop reading records.
- In MapReduce, RecordReader load data from its source and it converts the data into key-value pairs suitable for reading by the mapper. RecordReader communicates with the inputsplit until it does not read the complete file. The MapReduce framework defines RecordReader instance by the InputFormat.

By, default; it uses TextInputFormat for converting data into key-value pairs.

TextInputFormat provides 2 types of RecordReader :

1) **LineRecordReader-** LineRecordReader in Hadoop is the default RecordReader that TextInputFormat provides. Hence, each line of the input file is the new value and key is byte offset.

2) **SequenceFileRecordReader**- It reads data specified by the header of a sequence file.